

Labwork 4 - CUDA Threads

Do Minh Quang - 2540095
University of Science and Technology of Hanoi

1 GPU 2D Block

This labwork focuses on converting an RGB image to grayscale on the GPU using 2D thread blocks and comparing its performance with 1D implementation. The experiment uses 3167×3167 RGB image.



Figure 1: RGB image.

The 2D implementation extends the 1D approach by organizing threads and blocks along two dimensions. Each thread processes one pixel, with its position determined by its coordinates within the block and grid. Unlike the 1D implementation, the 2D version directly processes the original image without flattening the input.

```
@cuda.jit
def rgb2gray_gpu_2d(rgb, gray):
    tidx = cuda.threadIdx.x + cuda.blockIdx.x * cuda.
        blockDim.x
    tidy = cuda.threadIdx.y + cuda.blockIdx.y * cuda.
        blockDim.y
    g = np.uint8((rgb[tidy, tidx, 0] + rgb[tidy, tidx,
        1] + rgb[tidy, tidx, 2]) / 3)
```

```
gray[tidy, tidy, 0] = gray[tidy, tidy, 1] = gray[
    tidy, tidy, 2] = g
```

2 Results

Both implementations are executed 10 times, and the average execution time and standard deviation are recorded. A warm-up execution is performed before measurement to reduce the impact of initial CUDA overhead. The comparison uses a 1D block size of 1024 threads and a 2D block size of (32, 32), which also contains 1024 threads per block. The 1D implementation takes an average of 0.0201 seconds, while the 2D implementation takes 0.0200 seconds. These results indicate nearly identical performance between the two configurations.

2.1 Impact of block size

The block size is varied and the corresponding execution times are recorded. The results are presented in Figure 2.

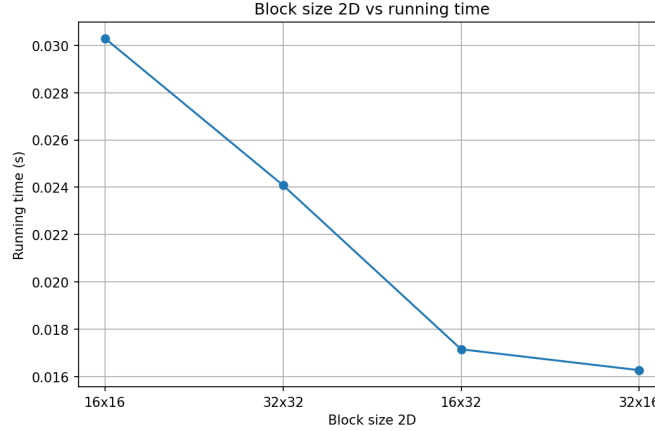


Figure 2: GPU execution time for different block sizes.

The results show that block size has a noticeable impact on execution time. The (16, 16) and (32, 32) configurations take approximately 0.03 and 0.024 seconds, respectively. In contrast, the (16, 32) and (32, 16) configurations achieve lower execution times of approximately 0.017 and 0.016 seconds.

The $(32, 16)$ configuration achieves the lowest execution time among the tested block sizes. Although $(32, 32)$ contains more threads per block, it does not provide better performance.